OPERATING SYSTEMS IN BANKING

Handling thousands of transactions simultaneously

Technical Presentation

Topics: suitable OS types • OS structure • system calls

JAYARANI V(192514115)

Lakshmi Devi)192525176

Jasmitha S (192524410)

1. Banking System Requirements

Why the operating system matters

High concurrency

Many users, ATMs, mobile apps and branches may submit requests at once.

Reliability

A transfer must not lose money or leave accounts in an inconsistent state.

Security & audit

Access control, isolation, logs and traceability are essential.

Low latency for balance checks and payments

Strong process and memory isolation

Safe storage, recovery and controlled access

2. Types of OS Suitable for Banking

A banking platform usually combines server operating systems with concurrency and reliability features

Multiuser OS

Supports many users and sessions at the same time. Useful for shared banking services.

Multiprocessing OS

Uses multiple CPU cores/processors to execute work in parallel and increase throughput.

Multitasking / Multithreading

Runs many tasks and request-handling threads concurrently; keeps services responsive.

Real-time OS (limited roles)

Useful where bounded response times matter, such as selected payment or device-control components—not necessarily the whole core bank.

Typical deployment: a secure, general-purpose server OS (often Linux or enterprise UNIX/Windows Server) configured for multiuser, multiprocessing and multitasking workloads.

3. Recommended OS Approach

The best choice is a resilient server platform, not one OS feature alone

Use an enterprise server OS with long-term security updates and strong access controls.

Deploy banking services across multiple servers or nodes for load sharing and failover.

Use process/thread scheduling, virtual memory, networking and file-system safeguards.

Pair the OS with a transactional database, encryption, monitoring, backups and disaster recovery.

Choose based on certification, support, workload, compliance and the bank’s existing architecture.

4. OS Structure Used

A layered view of a typical banking server

Banking applications	Web / mobile banking APIs • account service • transfer service
Middleware & database	Request routing • transaction manager • database engine
System-call interface	Controlled requests for files, processes, memory and network I/O
Kernel	Scheduler • memory manager • networking • file systems • device drivers
Hardware / virtual infrastructure	CPU • RAM • storage • network • virtual machines

In practice, modern server OSs commonly use a monolithic or modular kernel design; the banking application itself is not part of the kernel.

5. OS Architecture: Kernel Design

What structure is commonly found in server operating systems?

Monolithic / modular kernel

Core services such as scheduling, memory, file systems and networking run in kernel space. Modular components can be loaded as needed. Common in Linux and traditional UNIX designs.

Microkernel concept

Keeps the kernel smaller and moves more services to user space. Can improve isolation, but may introduce communication overhead. Used in some specialized systems.

For a typical bank server, a mature enterprise OS is selected for supportability, security, performance and operational tooling.

Architecture alone does not guarantee safety: configuration, patching, application design and database transaction controls also matter.

6. What Are System Calls?

The controlled interface between an application and the OS kernel

Banking application

↓ **system-call request**

OS kernel

↓ **result / error code**

Application continues

File & storage

open, read, write, fsync

Networking

socket, send, recv

Process control

fork, exec, wait

Synchronization

mutex/futex and related primitives

Exact system-call names vary by OS; many database operations are handled through libraries, database protocols and the database engine.

7. System Calls in a Bank Transaction

Example: transferring money between two accounts

1	**Receive request**	Banking service receives a transfer request over the network.
2	**Validate & authorize**	Application checks identity, permissions, limits and input.
3	**Database transaction**	Database begins a transaction and checks account state.
4	**Read / write / persist**	Storage and network I/O rely on OS services; database writes debit and credit.
5	**Commit or rollback**	Database commits atomically or rolls back on failure.
6	**Respond & audit**	Service returns the result and records audit information.

8. Concurrency, Consistency & Recovery

System calls support the work, while the database enforces transaction semantics

Concurrency

Scheduling, networking and synchronization allow many requests to progress at once.

Durability

Write and synchronization operations help storage layers persist data; database logging and commit rules provide transaction durability.

Failure handling

Errors from I/O, process exits or network timeouts must be handled safely; retries need idempotency controls.

ACID properties—atomicity, consistency, isolation and durability—are primarily implemented by the database/transaction layer, not by system calls alone.

Redundancy, backups, replication and tested recovery procedures help keep banking services available.

9. Security & Reliability Controls

Essential controls for financial transaction workloads

Least privilege: services run with only the permissions they require.

Authentication and authorization: verify users and service identities.

Isolation: separate processes, workloads and sensitive data.

Encryption: protect data in transit and at rest using approved mechanisms.

Auditability: centralize security and transaction logs; protect them from tampering.

Resilience: monitoring, patching, backups, failover and disaster-recovery tests.

CONCLUSION

A banking system needs more than speed.

- Multiuser, multitasking and multiprocessing capabilities support high transaction volume.
- A secure, reliable enterprise server OS provides the foundation.
- System calls connect applications to kernel services for networking, storage and process control.
- The database transaction layer—not system calls alone—ensures safe debit/credit behavior.

Thank you

References

Suggested sources for further reading

Operating System Concepts — Abraham Silberschatz, Peter B. Galvin and Greg Gagne.

Linux man-pages project — system calls and interfaces: man7.org/linux/man-pages/

Linux kernel documentation: docs.kernel.org

PostgreSQL documentation — transactions and concurrency control: postgresql.org/docs/

Oracle Database Concepts — transactions and data consistency: docs.oracle.com

Note: The examples are conceptual. Actual banking deployments vary by institution, platform and compliance requirements.